If we take the inheritance-based OCP example from above, we can write a function designed to use the `Vehicle` type like this:

```
function useVehicle(Vehicle $vehicle): void
{
    print "This vehicle can carry {$vehicle->getMaxPassengers()} people" .
    PHP_EOL;
}

useVehicle(new Vehicle());
useVehicle(new Car());
```

```
Output:
This vehicle can carry 0 people
This vehicle can carry 4 people
```

As you can see, `useVehicle()` functions perfectly correctly even when an instance of `Car` is passed into it, because `Car` is a subclass of `Vehicle`. When overloading parent members, it is vital that the overloaded members behave in the same way. The internal implementation can, of course, change, but input and output data types should remain identical.

There are in fact some mechanisms in the PHP object model to prevent violations of the LSP. For example, if we attempt to change the visibility of `getMaxPassengers()` to `private` (which would change the interface of the class), we get an error.

```
class Car extends Vehicle
{
    protected int $maxPassengers = 4;
    private function getMaxPassengers(): int
    {
        return $this->maxPassengers;
    }
}
```

```
Output:
Fatal error: Access level to Car::getMaxPassengers() must be public (as in
class Vehicle)
```

Likewise, we cannot change the return type.